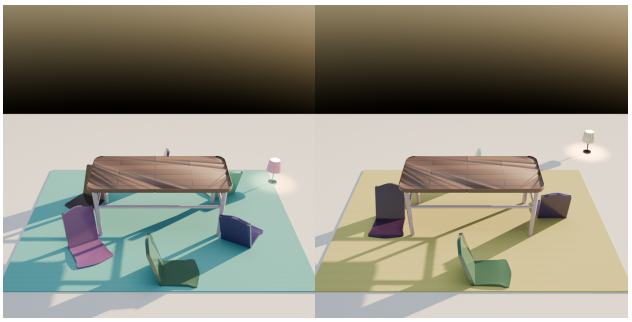
COMPOS3D: Your Reasoning LLM is Secretly a 3D Scene generator

Rishit Dagli [1] **Chau Nguyen** [2] **Avni Agrawal** [3]

Abstract

High-quality 3D scene data is expensive to produce yet essential for robotics simulation, autonomous driving, and virtual training. Existing approaches force a choice between visually diverse but uncontrollable generative models and expert-authored procedural systems that do not scale. We present Compos3D, a framework that bridges this gap by adapting the HypoGeniC hypothesis-discovery methodology from text classification to generative 3D scene synthesis. Rather than generating geometry end-to-end, Compos3D maintains a bank of natural-language design hypotheses per room type, discovered inductively from scene examples derived from SpatialLM. This formulation lets the system repurpose the large-scale pretraining of LLMs for 3D scene design without any geometry-specific fine-tuning. During training, a UCB-style bandit selects candidate hypotheses, an LLM compiles the selected hypotheses and prompt into a typed SceneProgram, and a heuristic or VLM-based critic scores the result. Failed generations accumulate in a per-room failure buffer; once the buffer reaches a threshold, the LLM proposes repair hypotheses that address observed failure modes, and the bank is updated accordingly. At inference time, the top-k hypotheses are retrieved and used to condition a single generation call; the resulting SceneProgram is then resolved into concrete object poses and rendered through Infinigen asset factories inside Blender. On a held-out set of 100 dining-room prompts, Compos3D wins 64% of pairwise VLM quality comparisons against a no-hypothesis baseline and achieves the same margin on layout coherence, demonstrating that learned design hypotheses substantially improve global scene organization.



Prompt: "A dining room with a dining table, several chairs, a lamp, and a rug"

Prompt: "A dining room with a dining table, four chairs, two lamps, and a rug"

Figure 1. COMPOS3D generates realistic procedural 3D scenes from text prompts and optionally images by reasoning over text. COMPOS3D can also edit existing scenes.

1. Introduction and Problem Statement

The demand for large-scale, high-fidelity 3D environments has accelerated in recent years, driven by robotics simulation, autonomous vehicle testing, and immersive mixed-reality experiences (Dosovitskiy et al., 2017; Deitke et al., 2022). However, production of such environments remains a significant bottleneck. Artist-authored scenes are expensive and difficult to scale; deep-learning based generative models produce visually plausible content but offer little structured control (Poole et al., 2022; Kerbl et al., 2023); and classic procedural systems require design rules to be hand-crafted by domain experts (Raistrick et al., 2023).

These approaches expose a fundamental trade-off between controllability and diversity. Diffusion-based methods, including diffusion NeRFs (Mildenhall et al., 2020) and 3D Gaussian Splatting (Kerbl et al., 2023), achieve high visual fidelity but function as black boxes: a user cannot easily isolate or modify specific attributes, such as repositioning a light source or enforcing furniture grounding constraints, without retraining or complex conditioning schemes. Procedural modeling tools such as Blender, Houdini, and VUE offer physical consistency and perfect reproducibility, but require expert authors to manually specify every design rule.

[1]dagliris [2]nguy2829 [3]agraw264. Correspondence to: <>.

Preprint. March 30, 2026.

This curation does not scale: every new scene type demands a fresh round of expert engineering.

Recent work has shown that large language models can produce structured scene representations from text prompts (Feng et al., 2023; Hollein et al., 2023), suggesting that LLM reasoning have the potential to substitute manual rule design. However, these approaches typically prompt the model once at test time, relying entirely on knowledge already present in the LLM's weights. They do not learn from scene data, cannot identify systematic failure modes, and produce no inspectable record of the design knowledge they apply.

We propose Compos3D, a framework that adapts the HypoGeniC hypothesis-discovery methodology (Liu et al., 2024) from text classification to generative 3D scene synthesis. The core idea is to treat design knowledge as an explicit, updatable bank of natural-language hypotheses—rules such as "chairs should be positioned within 50–80 cm of the nearest table surface and face inward toward it"—that are discovered inductively from scene data during training and applied at inference time without further model updates. Because the hypotheses are expressed in natural language, they are inspectable, editable, and composable: properties that pure neural generative models do not offer. Crucially, this formulation allows us to repurpose the large-scale pretraining of LLMs for a new problem domain, leveraging their ability to reason over natural-language descriptions of scenes without requiring any 3D-specific fine-tuning.

The central design principle is the separation of reasoning from rendering. Rather than asking a model to hallucinate geometry end-to-end, Compos3D first asks the model to articulate what makes a scene realistic—producing a typed JSON SceneProgram—and then executes those articulations deterministically through a procedural Blender backend built on Infinigen asset factories (Raistrick et al., 2023). During training, a multi-armed bandit loop selects candidate hypotheses, generates ScenePrograms conditioned on them, scores the results with a heuristic or VLM-based critic, and updates hypothesis utility estimates. Scenes that fall below a success threshold accumulate in a failure buffer; once the buffer reaches a threshold size, the LLM proposes corrective repair hypotheses that address the observed failure modes. At inference time, the hypothesis bank is frozen, and the top-k hypotheses for the inferred room type are retrieved and used to condition a single generation call.

The primary contributions of this work are:

- A formulation of 3D scene generation as iterative hypothesis discovery and execution, adapting the HypoGeniC framework (Liu et al., 2024) to a multimodal generative setting.
- A UCB-style multi-armed bandit training loop that balances exploration and exploitation over a growing hypothesis bank, using a lightweight heuristic critic or a VLM-based critic as a proxy reward signal.
- A repair-based regeneration mechanism that converts systematic failure modes into targeted hypothesis generation prompts, enabling self-improving scene quality over training.
- A fully deterministic procedural renderer that executes JSON SceneProgram through Infinigen factories inside Blender, ensuring physical plausibility and reproducibility.
- A scalable data processing pipeline derived from SpatialLM (Mao et al., 2025) and a cloud deployment built on AWS EC2, S3, and Terraform.

2. Related Work

2.1. 3D Generation with Autoregressive and Diffusion Models

Recent progress in 3D generation has been driven by both diffusion-based optimization and amortized 3D foundation models. A first line of work lifts powerful 2D priors into 3D by optimizing an explicit representation so that rendered views remain consistent with text or image conditions. DreamFusion introduced score distillation for text-to-3D generation, Magic3D improved this paradigm with a coarse-to-fine high-resolution mesh pipeline, and Text2Room extended multi-view diffusion guidance to room-scale scene reconstruction from synthesized views (Poole et al., 2022; Lin et al., 2023; Hollein et al., 2023). These methods established the viability of prompt-conditioned 3D synthesis, but they typically rely on expensive per-scene optimization loops.

A second line of work moves toward feed-forward 3D generation. Shap-E learns conditional implicit 3D representations from text or images (Jun & Nichol, 2023), LGM predicts 3D Gaussian representations from multi-view features (Tang et al., 2024), and CAT3D uses multi-view diffusion to create 3D scenes from one or more images (Gao et al., 2024). More recent large-scale systems emphasize higher-fidelity asset generation through structured latents and native 3D representations, including TRELLIS, TRELLIS.2, and Hunyuan3D 2.0 (Xiang et al., 2024; 2025; Tencent Hunyuan3D Team, 2025). In parallel, autoregressive models have become increasingly competitive for explicit mesh generation. MeshGPT tokenizes local mesh structure into triangle sequences (Siddiqui et al., 2024), while MeshAnything and MeshAnything V2 cast artist-quality mesh construction as shape-conditioned autoregressive generation with increasingly compact mesh tokenizations (Chen et al., 2024a;b). Despite their rapid progress in topology, texture, and recon-

struction fidelity, these methods are primarily designed for object- or asset-level generation and usually emit final geometry directly rather than an interpretable scene plan. Compos3D instead targets room-scale composition and treats language as the medium for explicit scene structure.

2.2. Procedural Scene Generation

Procedural generation offers a different route to scalable 3D content creation by expressing worlds through rules, parameterized asset factories, and constraint-based assembly. In simulation and embodied AI, systems such as CARLA and ProcTHOR demonstrate the value of procedurally generated environments for scale, coverage, and reproducibility (Dosovitskiy et al., 2017; Deitke et al., 2022). In graphics and data generation, Infinigen showed that large photorealistic outdoor worlds can be generated entirely procedurally without external asset libraries, while Infinigen Indoors extended the same philosophy to furniture, architecture, and indoor layout synthesis with procedural assets and arrangement solvers (Raistrick et al., 2023; 2024). The strength of this family is precise control over scene validity, rendering, and simulation; the weakness is that the scene-design logic itself must typically be written by experts.

Compos3D is closest to this procedural line on the execution side. Like Infinigen Indoors, we rely on Blender-based procedural assets and structured placement logic, but we do not assume that the semantic design rules are fixed a priori. Instead, we learn reusable natural-language hypotheses from scene data and compile them into an executable SceneProgram. In this sense, Compos3D sits between hand-authored procedural synthesis and end-to-end learned 3D generation: it inherits the controllability and fidelity of procedural rendering while replacing manually curated design rules with a learned symbolic planning layer.

2.3. 3D Generation with Language Models

Large language models are increasingly used as planners, programmers, and structured controllers for visual generation. LayoutGPT showed that in-context prompting can turn LLMs into visual planners that map language into structured 2D and 3D layouts (Feng et al., 2023). Open-Universe and Holodeck extend this idea to indoor scene generation by using language models to synthesize layout programs, object sets, and spatial constraints before layout optimization and asset retrieval (Aguina-Kang et al., 2024; Yang et al., 2024). AnyHome introduces a hierarchical structured representation for open-vocabulary house-scale generation and editing (Fu et al., 2024b), while SceneTeller combines language-based layout prediction, CAD retrieval, and Gaussian-splatting-based stylization to produce editable indoor scenes from natural-language descriptions (Öcal et al., 2024). These works establish language as a strong front-end for scene planning, but they largely rely on test-time prompting or task-specific pipelines rather than learning a persistent bank of reusable design knowledge.

A second thread brings language and multimodal models closer to inverse graphics and program synthesis. SceneLLM equips language models with 3D scene representations for understanding and planning in interactive indoor environments (Fu et al., 2024a). VIGA goes further and frames scene reconstruction and editing as an agentic write→run→render→compare→revise loop over executable graphics programs (Yin et al., 2026). Compos3D is most closely related to this language-driven family, but differs in what it stores and improves over time. Rather than relying only on one-shot prompting or fully trajectory-level self-correction, Compos3D learns an explicit bank of natural-language hypotheses, scores them with a UCB-style update rule, and augments them through failure-driven repair. The resulting SceneProgram is therefore not merely a temporary planning artifact, but the product of a reusable retrieval-and-revision loop that connects language reasoning to procedural rendering.

3. Methods

Compos3D separates scene synthesis into three parts: a learned bank of textual design hypotheses, a language model that compiles prompts and selected hypotheses into a structured SCENEPROGRAM, and a deterministic procedural renderer that turns the program into a 3D scene. This decomposition is the key adaptation from HypoGeniC-style hypothesis learning to generative 3D: the learned object is not a label predictor, but a reusable set of design priors whose value is measured only through generated scenes and critic feedback as shown in Figure 5.

Given a prompt p, Compos3D maintains a bank $\mathcal{H}_r$ of natural-language design hypotheses for each room type r, selects a subset $H \subseteq \mathcal{H}_r$, compiles (p, H) into a typed intermediate representation called a SCENEPROGRAM, and deterministically renders that program into a 3D scene. Because hypotheses are ordinary text, the system can reuse the large-scale pretraining of LLMs to reason about scene composition without any geometry-specific fine-tuning. Algorithm 1 shows the full training loop.

3.1. Scene Representation and Dataset

The core intermediate is a JSON SCENEPROGRAM

$$z = (p,\ r,\ s,\ H,\ A,\ C,\ \rho),$$

where s is an optional style tag, A is a list of asset specifications, C a list of textual placement constraints, and ρ a rendering specification. Each asset specification carries a category, an integer count, a free-form placement descrip-

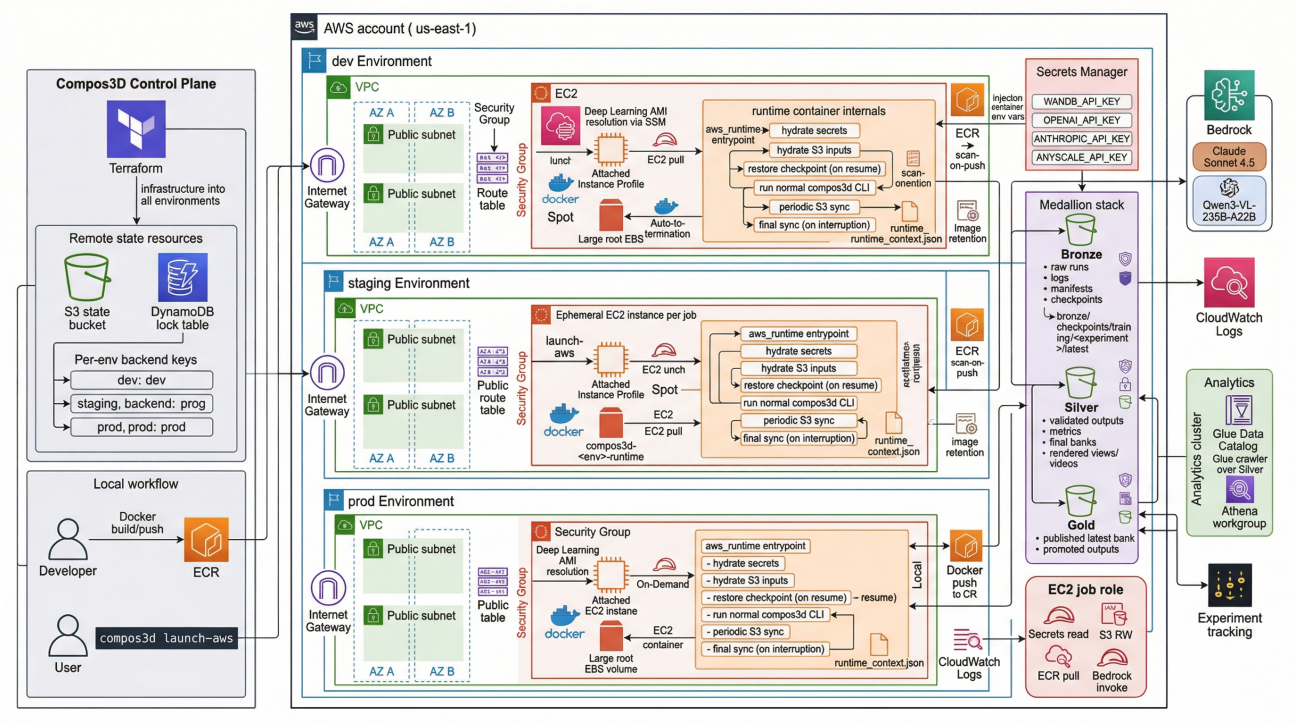


Figure 2. **AWS-native Compos3D architecture.** Terraform provisions isolated `dev`, `staging`, and `prod` environments, each with its own VPC, ECR repository, Secrets Manager entries, CloudWatch log group, IAM roles, and Bronze/Silver/Gold S3 buckets. At launch, an EC2 job runner pulls the runtime image from ECR, hydrates secrets and S3 inputs, restores checkpoints for resume when needed, runs the standard Compos3D command inside the container, and continuously syncs outputs and checkpoints back to S3 while sending logs to CloudWatch and metrics/media to Weights & Biases.

tion, and a rationale. Room types come from a three-class indoor ontology (dining room, living room, bedroom); assets are restricted to room-specific catalogs. Every generated SceneProgram is validated before rendering: the room type must match the request, all assets must belong to the catalog, counts must be positive integers, and each asset must carry both a placement description and a rationale. Programs that fail any check are rejected.

Training data are derived from SpatialLM (Mao et al., 2025) by mapping source room labels into the Compos3D room ontology, remapping raw object names into the asset catalog, retaining one canonical layout per room, and synthesizing a prompt from the resulting object counts. Each training example $d_i = (r_i, p_i, \mathcal{A}_i)$ carries a room type, prompt, and required asset set. This preprocessing is intentionally lossy: only room identity, prompt semantics, and required assets are retained, so the system must discover reusable design rules rather than memorize layouts.

3.2. Hypothesis Discovery and Training

Initialization. For each room type r, the hypothesis bank $\mathcal{H}_r$ is seeded by prompting an LLM on a small set of examples to produce short, abstract design rules. Each hypothesis h_i is immediately evaluated by generating a SCENEPROGRAM conditioned on h_i alone, scoring it against the seed examples, and recording the visit count n_i, mean critic score $\bar{s}_i$, and empirical accuracy $a_i = |\text{successes}|/n_i$.

UCB training loop. Unlike hypothesis learning for classification, our supervision signal is not a label but a scored scene. A hypothesis is useful only if conditioning on it improves the generated SCENEPROGRAM and its rendered realization. Training therefore proceeds through a full generate–render–score loop. At each step t, the system selects the top-k hypotheses for the current example's room type using the UCB-style reward

$$R_i(t) = a_i + \alpha \sqrt{\frac{\log t}{n_i}},$$

where α controls the exploration–exploitation tradeoff and the bonus is omitted when $t \leq 1$. Each selected hypothesis is evaluated independently by generating a SCENEPROGRAM conditioned on it alone. We then run a second, joint generation step using all selected hypotheses together. This distinction is important: the bank is updated using the marginal utility of individual hypotheses, while the system prediction logged for the task is the joint scene produced by the selected set. After each individual evaluation, n_i, a_i, $\bar{s}_i$, and R_i are updated.

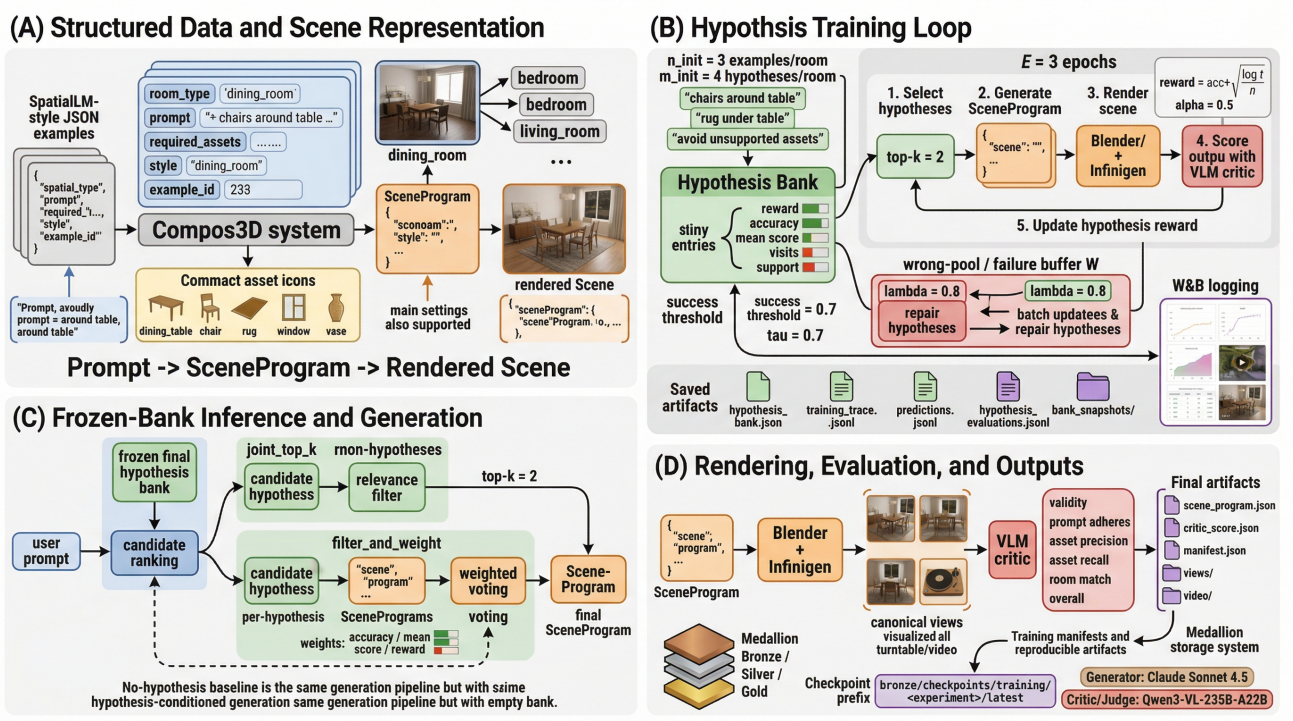


Figure 3. **Overview of Compos3D.** Starting from structured indoor-scene examples, Compos3D learns a bank of reusable textual hypotheses that capture object-layout regularities for scene generation. During training, the generator proposes hypothesis-conditioned SCENEPROGRAMS, renders them with Blender/Infinigen, and scores the resulting scenes with a VLM critic; these scores update a UCB-style reward that balances exploitation and exploration while failed cases trigger hypothesis repair. At inference time, a frozen hypothesis bank is adaptively filtered and aggregated to guide generation from a new prompt, producing a structured SCENEPROGRAM that is rendered into the final scene

Scoring is performed by a heuristic critic that compares the SceneProgram against the reference asset set $\mathcal{A}_i$,

$$\text{score}(z) = 0.25\, v + 0.35\, \text{pa} + 0.20\, \text{prec} + 0.20\, \text{rec},$$

where v is schema validity, pa is prompt adherence (derived from asset precision, asset recall, and room match), and prec/rec are asset precision and recall against $\mathcal{A}_i$. When rendered images are available, a VLM critic replaces the heuristic, scoring the images jointly against the SceneProgram.

Repair-based regeneration. Failed generations are treated as training signal rather than terminal errors. An example is added to a room-specific failure buffer when the number of selected hypotheses that score below the success threshold exceeds

$$\omega_t = \lambda \frac{|H_t|\, t}{|D|},$$

where $|H_t|$ is the number of selected hypotheses, $|D|$ the dataset size, and λ a scale parameter. This threshold is permissive early in training and tightens as the bank matures. Once the buffer reaches the target size $w_{\max} = b\, u$, the LLM is prompted with buffered failures to generate repair hypotheses: corrective rules that address missing required assets, wrong room semantics, or implausible placements. Repair hypotheses are evaluated on the failure buffer, deduplicated against $\mathcal{H}_r$, and merged; the bank is then truncated to its capacity by reward rank. This closes the loop between critic-identified failure modes and future generations.

3.3. Inference and Rendering

Hypothesis retrieval. Given a new prompt p, the system infers the room type r and retrieves candidate entries from $\mathcal{H}_r$, ranked by reward R_i, empirical accuracy a_i, prompt-asset overlap, and breadth of support across training examples. Our main inference rule is *filter-and-weight*: the system first filters the candidate hypotheses to those materially relevant to p, generates one candidate SCENEPROGRAM per surviving hypothesis, and then merges candidates by weighted voting, with weights derived from training accuracy and critic score. This preserves prompt-mentioned assets while allowing the bank to contribute reusable priors about layout and composition. We also evaluate a simpler *joint top-k* mode that passes the selected hypotheses directly to a single generation call.

Layout resolution. The SCENEPROGRAM is interpreted by a procedural layout module that resolves placement descrip-

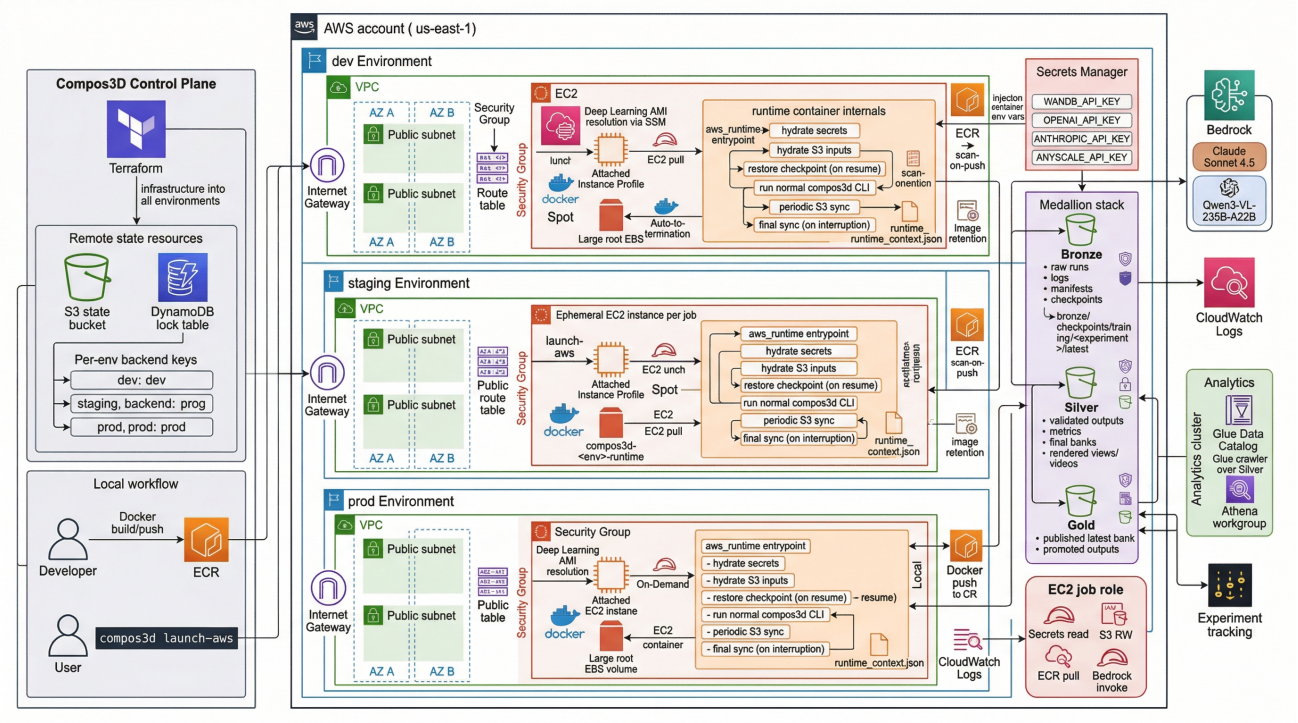


Figure 4. **AWS-native deployment.** The same inner Compos3D CLI runs locally or inside an ephemeral GPU EC2 job container. Runtime secrets are injected from a secret manager, checkpoints are synchronized to S3 for resume, logs stream to CloudWatch and Weights & Biases, and artifacts are written into Bronze, Silver, and Gold storage layers for reproducible training and evaluation.

tions into concrete object poses using room-specific anchor templates and relational rules. Dining tables and sofas serve as central anchors; chairs are arranged around the table; rugs are placed beneath the table or in front of the sofa; lamps are positioned adjacent to anchor furniture; vases sit on supporting surfaces with a positive vertical offset. Qualitative style cues in placement descriptions are mapped to alternative Infinigen (Raistrick et al., 2023) factory presets, allowing the LLM's symbolic output to influence geometric realization.

Rendering. Asset instances are spawned through Infinigen factories inside Blender with deterministic seeds. The renderer adds sky and area lighting and produces four canonical views: overhead and three oblique angles. For qualitative outputs, an orbital turntable video is also rendered. Video rendering is skipped during training to reduce wall-clock time.

3.4. AWS Infrastructure and Deployment

Compos3D runs locally or on AWS without modifying any training or inference logic, as illustrated in Figure 4. Cloud resources are provisioned with Terraform; jobs run as Docker containers on ephemeral GPU spot instances (G5 family) launched on demand. A container-side runtime wrapper fetches any S3-backed inputs, injects secrets from a secrets manager, and then invokes the standard training or inference command. For training jobs, checkpoints are periodically uploaded to S3 so that a preempted spot instance can be resumed from the latest snapshot. Persistent artifacts are organized as a three-tier data lake: raw traces and checkpoints in a bronze layer, structured query-ready data in a silver layer, and finalized hypothesis banks in a gold layer.

4. Experiments, Key Metrics, and Results

4.1. Experimental Setup

Datasets. We construct our evaluation dataset from SpatialLM (Mao et al., 2025), which provides structured prompts for indoor scene understanding and synthesis. All experiments in this section focus on dining rooms. We train on data derived from 60 prompts and evaluate on a split of 100 held-out prompts. We additionally reserve 24 validation prompts for hyperparameter tuning.

Baselines. We setup a *Baseline (no hyp.)* model, which invokes the same generator with an empty hypothesis bank. This isolates the contribution of learned hypotheses while keeping the underlying generator fixed. We report quantitative generation results on the 100-prompt held-out test set. We assess perceptual quality with a pairwise VLM judge

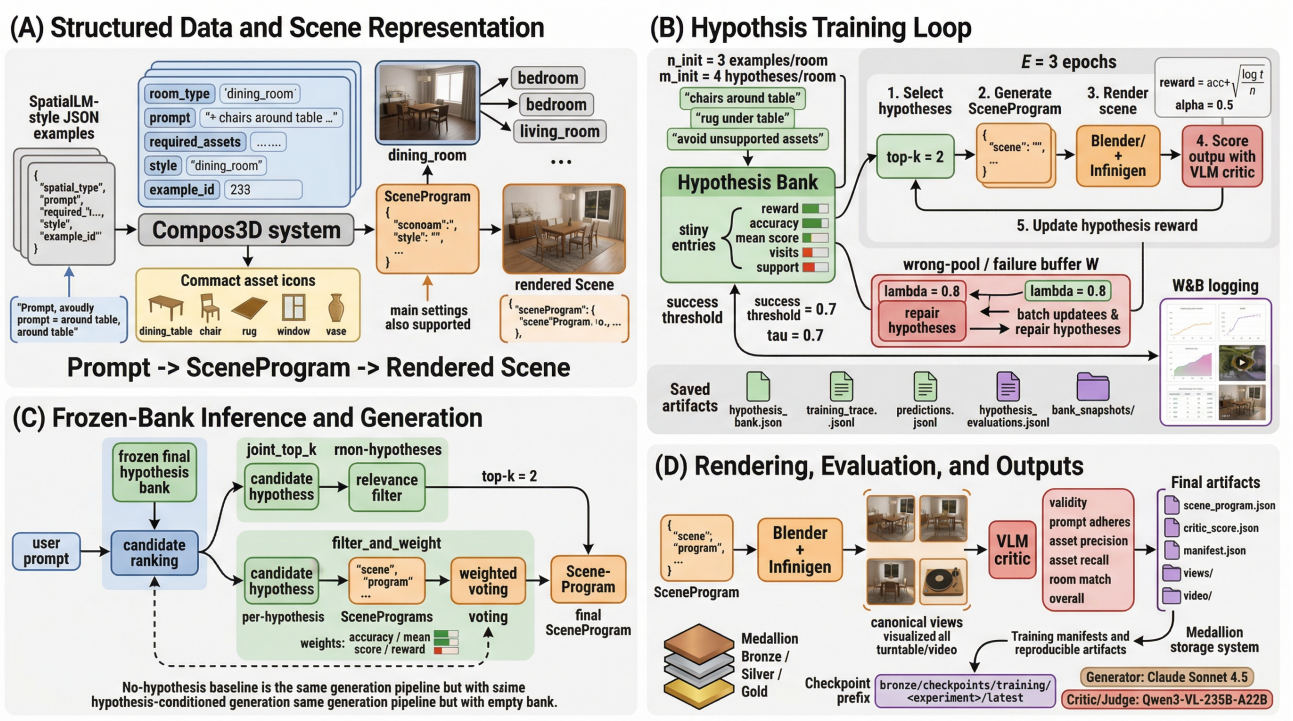


Figure 5. **Overview of Compos3D.** Seed examples initialize a room-specific hypothesis bank; a UCB-style loop selects hypotheses, evaluates them through scene generation, rendering, and critic feedback, and triggers repair when failures accumulate. At inference time, the frozen bank conditions scene generation, and the resulting SCENEPROGRAM is executed by the procedural Blender/Infinigen backend.

over 50 prompt-matched test pairs.

Table 1. **Implementation details.** Hyperparameters for our model.

Parameter	Value
Generator	Claude Sonnet 4.5
Scene critic	Qwen3-VL-235B-A22B
Pairwise VLM judge	Qwen3-VL-235B-A22B
Training Epochs (E)	3
Init. examples / room (n_{init})	3
Init. hypotheses / room (m_{init})	4
Max bank size (M)	10
Inference top-k	2
UCB exploration α	0.5
Wrong-pool scale (λ)	0.8
Success threshold (τ)	0.70
Update batch size (b)	2
Hypotheses updated / batch	2
Render resolution	256×256

We present additional implementation details in Table 1.

4.2. Metrics

Pairwise VLM quality. Our primary scene-quality metric is a pairwise VLM judge over prompt-matched image comparisons, following the preference-based evaluation paradigm of prior works (Lu et al., 2024; Yang et al., 2025). This metric asks which of two renders for the same prompt is better overall.

Layout coherence. We additionally ask the pairwise VLM judge to compare global spatial organization. This metric measures whether a scene reads as a coherent dining room rather than as a collection of individually plausible but poorly composed objects.

Requested-asset F1. Since COMPOS3D produces an explicit symbolic scene program, we can measure requested-asset precision and recall directly from the predicted asset set and report their harmonic mean. This is our primary controllability metric and is aligned with the object-level evaluation perspective used in structured indoor scene generation (Lin & Mu, 2024; Yang et al., 2024).

Exact asset-set match. This metric reports the fraction of examples for which the predicted set of requested assets matches the prompt exactly. It is stricter than asset-level F1 because a single missing or extra requested asset causes failure.

Hallucinated asset rate. We measure the rate at which the generated scene includes assets that were not requested by the prompt. This captures a common failure mode in scene

synthesis: visually plausible but semantically unsupported additions.

Mean hypothesis score. For training-dynamics analysis, we track the mean internal hypothesis score across bank snapshots. This summarizes how the bank's aggregate utility evolves throughout training.

Mean UCB reward. We additionally report the mean UCB-style reward assigned to hypotheses during training, following the bandit-style selection objective that governs bank maintenance (Auer et al., 2002).

Mean training accuracy. We report average per-hypothesis training accuracy across bank snapshots to track predictive reliability during training.

Failure taxonomy counts. Finally, we aggregate critic notes and failed examples into counts over major error categories, which we use to summarize the dominant remaining failure modes of the trained system.

4.3. Generation Results

In Table 2 and Figure 6, we show that learned hypotheses substantially improve visual generation quality. On the primary pairwise VLM metric, COMPOS3D wins 64% of prompt-matched comparisons against Baseline (no hyp.), and it achieves the same margin on layout coherence. COMPOS3D is better at coordinating tables, chairs, and lamps into globally coherent layouts rather than producing scenes that are locally plausible but compositionally weaker.

Baseline (no hyp.) is stronger on asset fidelity, achieving higher requested-asset F1 and exact asset-set match while hallucinating fewer objects. This suggests that the generator alone is already competent on many easy prompts, whereas the learned hypothesis bank is most valuable when global scene organization matters more than strict symbolic matching.

4.4. Editing Results

Instead of synthesizing a scene from scratch, we task our model to perform a targeted modification while preserving the rest of the composition. In Figure 6, we show the results of the first "add-rug" edit. COMPOS3D inserts the requested rug while largely preserving the original table-and-chair arrangement; Baseline (no hyp.) and the ablations either do not add the rug or change the other objects. In the second "remove-lamp" edit, COMPOS3D similarly removes the targeted object while keeping the remaining room layout since our method extensively reasons in text before performing the edit.

4.5. Ablations

We perform several ablations in Figure 2 and Table 2. The largest degradation comes from removing repair or replacing learned selection with random choice, suggesting that bank maintenance and informed hypothesis selection matter more than small changes in the final inference rule once the bank has matured. This is because the bank provides the dominant structural signal, while adaptive inference determines how effectively that signal is used on a particular prompt.

4.6. Hyperparameter Sensitivity and Training Dynamics

Panels (e)–(g) in Figure 7 show a broad optimum near $\alpha = 0.5$, $\lambda = 0.8$, and $\tau = 0.70$. Panels (a)–(d) in Figure 7 show that the bank expands early, then shifts toward higher-reward and higher-accuracy hypotheses, which is the expected behavior under UCB-style selection (Auer et al., 2002). Figure 8 shows the same trend from a single scalar: mean hypothesis score rises over the run and ends substantially above its initial value.

Panel (h) in Figure 7 shows the main remaining failure modes. Lamp placement and rug handling dominate, followed by chair arrangement and count-sensitive table errors. The next improvement should therefore come from better spatial hypotheses for lighting, rugs, and window-conditioned room structure rather than from simply increasing bank size.

5. Team reflection on the project including connection to the course material

Our team's primary motivation with Compos3D was to bridge the gap between high-level creative intent and the technical precision required for 3D environments. Particularly, we wanted to use the concepts taught in this course on using LLMs, reasoning with LLMs, and deploying them to solve this problem. We also learned about RL and we built a new method that is in spirit very close to RL but for hypothesis discovery.

Throughout the semester, we often discussed the limitations of large language models, specifically their tendency toward hallucination, which we saw as a major blocker for generating physically valid indoor scenes. This directly informed our decision to adopt the principles of constrained decoding covered in Lecture 4. Rather than letting the model output free-form descriptions, we engineered a strict SceneProgram schema using Pydantic. This acted as a "compiler" for the LLM's creativity, ensuring that every asset count and placement hint was validated before it ever reached the rendering pipeline.

Deploying this system and making a real-time demo required us to think about the "Infrastructure as Code" dis-

Table 2. **Generation Results.** We report the results of the held-out generation prompts. The Pairwise VLM Quality is computed between a given setting and COMPOS3D except for the COMPOS3D row where we compute the score against Baseline (no hyp.).

Method	Pairwise VLM Quality [↑]	Layout Coherence [↑]	Requested Asset F1 [↑]	Exact Asset Set Match [↑]	Hallucinated Asset Rate [↓]
COMPOS3D	0.6400	0.6400	0.9441	0.7100	0.0925
Baseline (no hyp.)	0.3600	0.3600	0.9625	0.7900	0.0630
Ablations					
Fixed hypotheses	0.4000	0.4200	0.9310	0.6820	0.1060
No repair	0.4400	0.4400	0.9370	0.6970	0.0990
Greedy selection	0.4600	0.4800	0.9410	0.7060	0.0950
Random selection	0.3000	0.3000	0.9180	0.6420	0.1210
Joint top-k	0.4800	0.4800	0.9400	0.7030	0.0960
Top-$k = 1$	0.4200	0.4400	0.9340	0.6910	0.1030

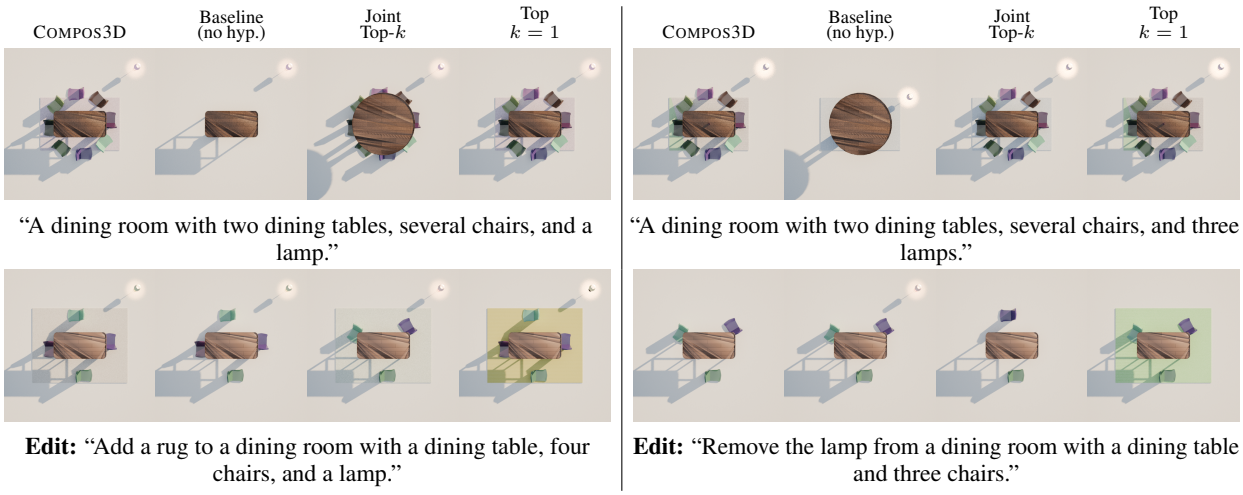


Figure 6. **Qualitative comparison.** We show two held-out generation prompts and two editing instructions.

cussed in Lecture 2. Because our pipeline involves heavy procedural rendering in Blender, we couldn't rely on local compute alone. We used Terraform to provision a professional-grade AWS stack, including ECR for our containerized environments and S3 for data persistence. We implemented a Medallion Architecture for our storage, splitting data into bronze, silver, and gold tiers, as suggested in the Lecture 9 discussion on scaling ML systems. This allowed us to manage the transition from raw SpatialLM (Mao et al., 2025) records to validated SceneProgram and, eventually, high-fidelity renders and metrics. Managing secrets through AWS Secrets Manager and monitoring runs via CloudWatch and Weights and Biases.

One of the challenges was implementing the hypothesis-guided training loop, which allowed us to apply the Reinforcement Learning concepts from Lecture 8 in a practical way. While we didn't use a full PPO algorithm, we treated the selection of layout design ideas as a Multi-Armed Bandit problem. By using UCB-style selection to pick design hypotheses and using a VLM-based critic to provide the reward signal, we effectively closed the loop between generation and feedback.

Finally, the project forced us to grapple with the "Build vs. Buy" dilemma from Lecture 3 when it came to evaluation. We had to decide whether to write our own heuristic scoring functions or rely on state-of-the-art Vision-Language Models (VLMs) as judges. We ultimately did both, realizing that while heuristics provide fast, cheap signals on validity, a VLM is necessary to capture the subjective feel of a well-designed room. This mirrored our class discussions on how to evaluate ML products when ground truth is hard to define. Ultimately, this project forced us to stop looking at the model in isolation and instead focus on the much harder task of engineering the entire system that makes those model outputs actually useful. It shifted our perspective from being model-centric to being system-centric.

6. Future Work and Next Steps

Compos3D already provides a strong base for controllable indoor scene generation by combining hypothesis-guided language modeling, structured scene programs, and procedural rendering. The next phase of the project should build on that foundation and move toward a broader system for procedural asset generation, larger-scale preference learn-

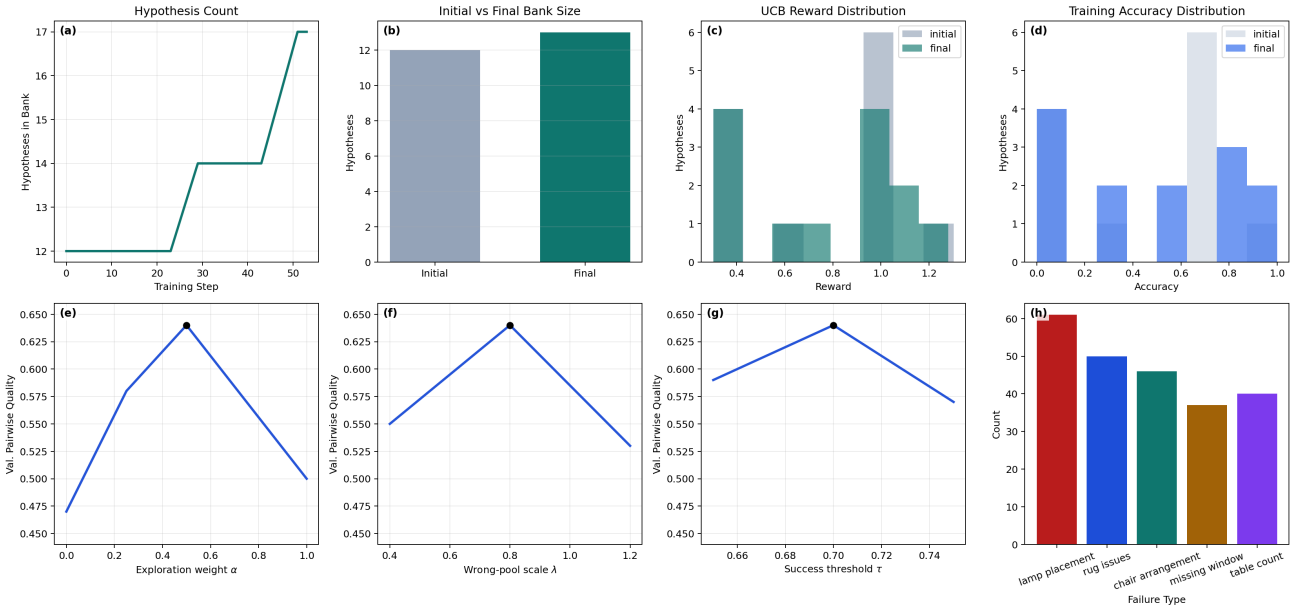


Figure 7. **Training diagnostics, validation sensitivity, and failure modes.** Panels (a)–(d) show hypothesis-bank evolution over training: bank size, initial-versus-final bank size, UCB reward distribution, and training accuracy distribution. Panels (e)–(g) show validation sensitivity to α, λ, and τ, with the selected operating point marked in black. Panel (h) summarizes the final failure taxonomy from critic notes and failed examples.

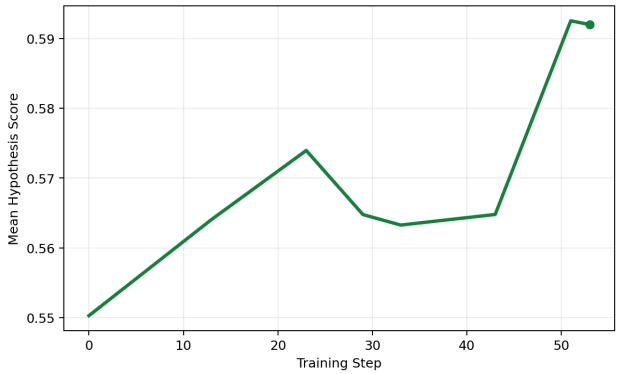


Figure 8. **Mean hypothesis score over training.** The score rises over the training run, indicating that the bank becomes more useful as training proceeds.

ing, simulation-ready geometry, and persistent 3D memory for video models. These directions are supported by recent work on hypothesis generation with LLMs, structured indoor scene modeling, procedural generation for simulation, and explicit 3D representations for video consistency (Liu et al., 2024; Mao et al., 2025; Joshi et al., 2025; Zhang et al., 2025).

6.1. LLM-generated procedural factories

One important next step is to move beyond using procedural assets as fixed building blocks and instead have the LLM help design the procedural factories themselves. Right now, Compos3D depends on a predefined asset ecosystem, which limits the range of objects and styles the system can express. A more ambitious approach would treat factory design as part of the generation problem, so the model can produce parameterized generators for new furniture, decor, or other scene elements. This fits well with broader work on LLMs as structured program generators, as well as recent procedural asset systems that rely on flexible, reusable generation pipelines (Huynh & Lin, 2025; Joshi et al., 2025).

6.2. Larger preference dataset

Compos3D would also benefit from training on a much larger and more diverse preference dataset. The current system can learn useful priors from a relatively small set of room types and examples, but broader coverage would make it easier to generalize across layouts, styles, and prompt variations. Recent structured indoor-scene work such as SpatialLM shows the value of scaling synthetic scene data for spatial reasoning and generation, while hypothesis-based systems such as HypoGeniC suggest that iterative refinement becomes more effective when the underlying evidence base is larger and more varied (Mao et al., 2025; Liu et al., 2024). In practice, a larger preference dataset would help the critic and hypothesis bank learn what makes a scene not just valid, but also coherent, aesthetically aligned, and robust across domains.

6.3. Automatic factory conversion

Another useful direction is to build an automated pipeline that converts existing procedural factories into a format that an LLM can work with more easily. Many procedural systems are powerful but awkward to integrate because they rely on handwritten code, complex dependencies, and a lot of implicit design choices. An automated "factory compiler" would pull out the essential parts of each factory (parameters, constraints, outputs, and semantic role) — and turn them into a modular schema that the model can query, compose, and adapt. That would make the system much easier to extend while still preserving the value of existing procedural codebases, and it follows the broader trend in LLM-based code structuring and translation research (Huynh & Lin, 2025).

6.4. Simulation-ready assets

Because procedural assets are represented as explicit geometry, they are especially well suited for simulation and interaction. That is a real advantage over systems that only produce visual outputs, since simulation depends on collision geometry, semantic labels, and physically meaningful structure. Recent work on procedural generation of articulated simulation-ready assets shows that this kind of approach can support robotics, physical interaction, and embodied learning at scale (Joshi et al., 2025). For Compos3D, this opens a clear path toward generating scenes that are useful not only for rendering, but also for simulation, manipulation, and other embodied tasks, which makes the project relevant to both graphics and robotics communities (Bonetto et al., 2025).

6.5. 3D memory for video generation

A further promising direction is to use Compos3D-style structured 3D representations as a kind of persistent memory for video generation. Video models often have trouble maintaining object consistency, spatial coherence, and stable scene structure over time, especially across long sequences or viewpoint changes. Recent work on world-consistent and scene-conditioned video generation suggests that explicit 3D modeling can help address this by giving the model a persistent representation of the environment (Zhang et al., 2025). Compos3D could therefore act as a scene-level memory system that stores layout, object identity, and spatial relationships, helping future video models generate longer sequences that stay more coherent and controllable.

Algorithm 1 COMPOS3D training loop. $S = \{e_i = (p_i, r_i, a_i)\}_{i=1}^{N}$ is the training set with prompt p_i, room type r_i, and required asset set a_i; G is the generator and C the critic; E is the number of epochs; n_{init} and m_{init} are the number of seed examples and initial hypotheses per room; k is the number of selected hypotheses; M is the maximum bank size per room; α is the UCB exploration weight; λ is the dynamic failure-pool scale; b is the update batch size; u is the number of repair rounds; m_{rep} is the number of repair hypotheses generated per round; τ is the success threshold; and t denotes the within-epoch sample index.

Require: training set $S = \{e_i = (p_i, r_i, a_i)\}_{i=1}^{N}$, generator G, critic C
Require: $E, n_{\text{init}}, m_{\text{init}}, k, M, \alpha, \lambda, b, u, m_{\text{rep}}, \tau$
Ensure: final banks $\{H_r\}$ and combined predictions $\mathcal{P}$

$\mathcal{P} \leftarrow \emptyset$
for each room type r **do**
  $S_r \leftarrow \{e \in S : e.r = r\}$
  $S_r^0 \leftarrow$ first n_{init} examples from S_r
  $\widetilde{H}_r \leftarrow G.\textsc{GenHyp}(r, S_r^0, m_{\text{init}})$
  $H_r \leftarrow \textsc{EvalInit}(\widetilde{H}_r, S_r^0, \tau)$
  $W_r \leftarrow \emptyset$
end for
for epoch $= 1, \ldots, E$ **do**
  for each $e_t = (p_t, r_t, a_t) \in S$ **do**
    if epoch $= 1$ and $e_t \in S_{r_t}^0$ **then**
      continue
    end if
    $H_t \leftarrow \textsc{SelectTopK}(H_{r_t}, k)$
    $n_{\text{wrong}} \leftarrow 0$
    for each $h \in H_t$ **do**
      $z_h \leftarrow G.\textsc{GenScene}(p_t, r_t, [h])$
      $I_h \leftarrow \textsc{Render}(z_h)$
      $c_h \leftarrow C.\textsc{Score}(z_h, I_h, e_t)$
      $\textsc{Update}(h, c_h.\text{overall}, t, \tau)$
      if $c_h.\text{overall} < \tau$ **then**
        $n_{\text{wrong}} \leftarrow n_{\text{wrong}} + 1$
      end if
    end for
    $T_t \leftarrow [h.\text{text} : h \in H_t]$
    $z_t \leftarrow G.\textsc{GenScene}(p_t, r_t, T_t)$
    $I_t \leftarrow \textsc{Render}(z_t)$
    $c_t \leftarrow C.\textsc{Score}(z_t, I_t, e_t)$
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{(e_t, H_t, z_t, c_t)\}$
    $\omega_t \leftarrow \lambda |H_t| t/N$
    if $|H_t| = 0$ or $n_{\text{wrong}} \geq \omega_t$ **then**
      $W_{r_t} \leftarrow W_{r_t} \cup \{e_t\}$
    end if
    if $|W_{r_t}| \geq b\,u$ **then**
      $N_t \leftarrow \emptyset$
      for $j = 1, \ldots, u$ **do**
        $\widetilde{H}^{(j)} \leftarrow G.\textsc{GenHyp}(r_t, W_{r_t}, m_{\text{rep}})$
        $N_t \leftarrow N_t \cup \textsc{EvalInit}(\widetilde{H}^{(j)}, W_{r_t}, \tau)$
      end for
      $H_{r_t} \leftarrow \textsc{TopM}(H_{r_t} \cup N_t, M)$
      $W_{r_t} \leftarrow \emptyset$
    end if
  end for
end for
return $\{H_r\}, \mathcal{P}$

References

Aguina-Kang, R., Gumin, M., Han, D. H., Morris, S., Yoo, S. J., Ganeshan, A., Jones, R. K., Wei, Q. A., Fu, K., and Ritchie, D. Open-universe indoor scene generation using llm program synthesis and uncurated object databases, 2024. URL https://arxiv.org/abs/2403.09675.

Auer, P., Cesa-Bianchi, N., and Fischer, P. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2):235–256, 2002.

Bonetto, E., Xu, C., and Ahmad, A. Grade: Generating realistic and dynamic environments for robotics. https://doi.org/10.1177/02783649251346211, 2025. Robotics-related environment generation paper.

Chen, Y., He, T., Huang, D., Ye, W., Chen, S., Tang, J., Chen, X., Cai, Z., Yang, L., Yu, G., Lin, G., and Zhang, C. Meshanything: Artist-created mesh generation with autoregressive transformers, 2024a. URL https://arxiv.org/abs/2406.10163.

Chen, Y., Wang, Y., Luo, Y., Wang, Z., Chen, Z., Zhu, J., Zhang, C., and Lin, G. Meshanything v2: Artist-created mesh generation with adjacent mesh tokenization, 2024b. URL https://arxiv.org/abs/2408.02555.

Deitke, M., VanderBilt, E., Herrasti, A., Weihs, L., Salvador, J., Ehsani, K., Han, W., Kolve, E., Farhadi, A., Kembhavi, A., and Mottaghi, R. Procthor: Large-scale embodied ai using procedural generation. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/27c546ab1e4f1d7d638e6a8dfbad9a07-Abstract-Conference.html.

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., and Koltun, V. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*, volume 78 of *Proceedings of Machine Learning Research*, pp. 1–16. PMLR, 2017. URL https://proceedings.mlr.press/v78/dosovitskiy17a.html.

Feng, W., Zhu, W., Fu, T.-j., Jampani, V., Akula, A., He, X., Basu, S., Wang, X. E., and Wang, W. Y. Layoutgpt: Compositional visual planning and generation with large language models. *arXiv preprint arXiv:2305.15393*, 2023. doi: 10.48550/arXiv.2305.15393. URL https://layoutgpt.github.io/.

Fu, R., Liu, J., Chen, X., Nie, Y., and Xiong, W. Scene-llm: Extending language model for 3d visual understanding and reasoning, 2024a. URL https://arxiv.org/abs/2403.11401.

Fu, R., Wen, Z., Liu, Z., and Sridhar, S. Anyhome: Open-vocabulary generation of structured and textured 3d homes. In *European Conference on Computer Vision*, pp. 52–70. Springer, 2024b. doi: 10.1007/978-3-031-72933-1_4. URL https://ivl.cs.brown.edu/research/anyhome.html.

Gao, R., Holynski, A., Henzler, P., Brussee, A., Martin-Brualla, R., Srinivasan, P. P., Barron, J. T., and Poole, B. Cat3d: Create anything in 3d with multi-view diffusion models. *Advances in Neural Information Processing Systems*, 2024. URL https://cat3d.github.io/.

Hollein, L., Cao, A., Owens, A., Johnson, J., and Nießner, M. Text2room: Extracting textured 3d meshes from 2d text-to-image models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 7909–7920, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Hollein_Text2Room_Extracting_Textured_3D_Meshes_from_2D_Text-to-Image_Models_ICCV_2023_paper.html.

Huynh, N. and Lin, B. Large language models for code generation. https://arxiv.org/html/2503.01245v2#S6, 2025. arXiv HTML preprint.

Joshi, A., Han, B., Nugent, J., Saez-Diez, M. G., Zuo, Y., Liu, J., Wen, H., Alexandropoulos, S., Kayan, K., Calveri, A., Sun, T., Liu, G., Shao, Y., Raistrick, A., and Deng, J. Procedural generation of articulated simulation-ready assets. https://arxiv.org/html/2505.10755v3, 2025. arXiv preprint.

Jun, H. and Nichol, A. Shap-e: Generating conditional 3d implicit functions. *arXiv preprint arXiv:2305.02463*, 2023. doi: 10.48550/arXiv.2305.02463. URL https://arxiv.org/abs/2305.02463.

Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023. URL https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/.

Lin, C. and Mu, Y. Instructscene: Instruction-driven 3d indoor scene synthesis with semantic graph prior. In *International Conference on Learning Representations (ICLR)*, 2024.

Lin, C.-H., Gao, J., Tang, L., Takikawa, T., Zeng, X., Huang, X., Kreis, K., Fidler, S., Liu, M.-Y., and Lin, T.-Y. Magic3d: High-resolution text-to-3d content creation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 300–309, June 2023. URL https://openaccess.

thecvf.com/content/CVPR2023/html/Lin_Magic3D_High-Resolution_Text-to-3D_Content_Creation_CVPR_2023_paper.html.

Liu, H., Zhou, Y., Li, M., Yuan, C., and Tan, C. Literature meets data: A synergistic approach to hypothesis generation. https://chicagohai.github.io/hypogenic-demo/, 2024. Accessed 2026-03-30.

Lu, Y., Jiang, D., Chen, W., Wang, W. Y., Choi, Y., and Lin, B. Y. Wildvision: Evaluating vision-language models in the wild with human preferences. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=i92eyFCQHC.

Mao, Y., Zhong, J., Fang, C., Zheng, J., Tang, R., Zhu, H., Tan, P., and Zhou, Z. Spatiallm: Training large language models for structured indoor modeling. 2025.

Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. Nerf: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. URL https://arxiv.org/abs/2003.08934.

Öcal, B. M., Tatarchenko, M., Karaoglu, S., and Gevers, T. Sceneteller: Language-to-3d scene generation. *arXiv preprint arXiv:2407.20727*, 2024. doi: 10.48550/arXiv.2407.20727. URL https://arxiv.org/abs/2407.20727.

Poole, B., Jain, A., Barron, J. T., and Mildenhall, B. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022. doi: 10.48550/arXiv.2209.14988. URL https://arxiv.org/abs/2209.14988.

Raistrick, A., Lipson, L., Ma, Z., Mei, L., Wang, M., Zuo, Y., Kayan, K., Wen, H., Han, B., Wang, Y., Newell, A., Law, H., Goyal, A., Yang, K., and Deng, J. Infinite photorealistic worlds using procedural generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12630–12641, June 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Raistrick_Infinite_Photorealistic_Worlds_Using_Procedural_Generation_CVPR_2023_paper.html.

Raistrick, A., Mei, L., Kayan, K., Yan, D., Zuo, Y., Han, B., Wen, H., Parakh, M., Alexandropoulos, S., Lipson, L., Ma, Z., and Deng, J. Infinigen indoors: Photorealistic indoor scenes using procedural generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 21783–21794, June 2024. doi: 10.1109/CVPR52733.2024.02058. URL https://openaccess.thecvf.com/content/CVPR2024/html/Raistrick_Infinigen_Indoors_Photorealistic_Indoor_Scenes_using_Procedural_Generation_CVPR_2024_paper.html.

Siddiqui, Y., Alliegro, A., Artemov, A., Tommasi, T., Sirigatti, D., Rosov, V., Dai, A., and Nießner, M. Meshgpt: Generating triangle meshes with decoder-only transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 19615–19625, June 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Siddiqui_MeshGPT_Generating_Triangle_Meshes_with_Decoder-Only_Transformers_CVPR_2024_paper.html.

Tang, J., Chen, Z., Chen, X., Wang, T., Zeng, G., and Liu, Z. Lgm: Large multi-view gaussian model for high-resolution 3d content creation. *arXiv preprint arXiv:2402.05054*, 2024. doi: 10.48550/arXiv.2402.05054. URL https://arxiv.org/abs/2402.05054.

Tencent Hunyuan3D Team. Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation, 2025. URL https://arxiv.org/abs/2501.12202.

Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., and Yang, J. Structured 3d latents for scalable and versatile 3d generation. *arXiv preprint arXiv:2412.01506*, 2024. URL https://arxiv.org/abs/2412.01506. CVPR 2025 Highlight.

Xiang, J., Chen, X., Xu, S., Wang, R., Lv, Z., Deng, Y., Zhu, H., Dong, Y., Zhao, H., Yuan, N. J., and Yang, J. Native and compact structured latents for 3d generation. *Tech report*, 2025. URL https://microsoft.github.io/TRELLIS.2/.

Yang, Y., Sun, F.-Y., Weihs, L., VanderBilt, E., Herrasti, A., Han, W., Wu, J., Haber, N., Krishna, R., Liu, L., Callison-Burch, C., Yatskar, M., Kembhavi, A., and Clark, C. Holodeck: Language guided generation of 3d embodied ai environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16227–16237, June 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Yang_Holodeck_Language_Guided_Generation_of_3D_Embodied_AI_Environments_CVPR_2024_paper.html.

Yang, Y., Jia, B., Zhang, S., and Huang, S. Sceneweaver: All-in-one 3d scene synthesis with an extensible and self-reflective agent, 2025.

Yin, S., Ge, J., Wang, Z. Z., Li, X., Black, M. J., Darrell, T., Kanazawa, A., and Feng, H. Vision-as-inverse-graphics agent via interleaved multimodal reasoning. *arXiv preprint arXiv:2601.11109*, 2026. URL https://arxiv.org/abs/2601.11109.

Zhang, Q., Zhai, S., Bautista, M. A., Miao, K., Toshev, A., Susskind, J., and Gu, J. World-consistent video diffusion with explicit 3d modeling. https://arxiv.org/html/2412.01821v1, 2025. arXiv preprint.